

StreamWeave: Reconciling Off-Policy Expert Supervision with Fully-Asynchronous Policy Learning

Anonymous submission

Abstract

Reinforcement learning with verifiable rewards (RLVR) spends substantial generation work on long rollouts for problems that the policy cannot yet solve, but obtains no group-relative learning signal when every response to a prompt fails. Fully-asynchronous RL reduces execution cost by overlapping rollout generation with model training, while expert trajectories provide solution paths where policy-generated signal is absent. In the group-conditioned setting studied here, however, the outcome of a complete group determines both the training source and the learning role of each sample, whereas a fully-asynchronous runtime executes trajectory attempts independently. Our key insight is that a complete group can remain the boundary of a learning decision without making its completion a pipeline-wide execution barrier. StreamWeave executes attempts independently, reconstructs a complete group at the source-decision boundary, and converts the resolved source and generation context into the corresponding training input. Policy and expert inputs are then consumed in one primary update with one policy objective and the same averaging rule, while rollout generation and model training remain concurrent. On mathematical reasoning benchmarks, StreamWeave achieves an average score of 38.5, comparable to 37.7 for a synchronous quality reference that uses the same group-success selector. It also reduces the processing time for 128 prompt groups from 46 to 28 seconds and improves prompt-group throughput by $1.64\times$ under the same GPU budget. StreamWeave preserves group-conditioned heterogeneous learning while realizing the efficiency of fully-asynchronous execution in one training system.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) improves the reasoning ability of large language models by reinforcing automatically verifiable successes. On problems that the policy cannot yet solve, however, it may spend substantial generation work without obtaining a learning signal. Group-based RLVR generates multiple rollouts from the same prompt to search for a correct solution, and long reasoning trajectories make this generation a major part of training wall-clock time. When every rollout in the group

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

fails, the reward contrast within the group vanishes, leaving no relative learning signal. In our training trace, all-failure groups account for 22.4% of all groups but 26.8% of generated response tokens. Computational burden and signal scarcity therefore concentrate in the same hard region. Fully-asynchronous RL reduces the execution bottleneck by overlapping rollout generation with model training, while methods that use expert trajectories provide external solution paths for problems that the policy fails to solve. Scaling group-conditioned RLVR needs both capabilities, but combining them does not by itself determine how a decision defined on a complete group reaches a runtime built to advance attempts independently.

The two capabilities interact directly because a complete-group outcome now does more than support a relative signal: a group containing a success learns from policy rollouts, whereas an all-failure group selects the paired expert trajectory. The group therefore determines both the training source and how the selected samples contribute to the update. Fully-asynchronous execution gains efficiency by running trajectory attempts from the same group independently. Treating a complete group as the execution unit places its slowest trajectory on the pipeline critical path, while losing source identity would erase the distinct learning roles of policy and expert samples. The central challenge is to carry a complete-group decision into an asynchronous training stream without binding trajectory execution back into groups. We therefore ask: **Can a system preserve complete-group decisions while retaining trajectory-level fully-asynchronous execution?**

Figure 1 summarizes how StreamWeave bounds the two dependencies required by this learning program. It executes trajectory attempts independently, reconstructs a complete group only for its source decision, and carries the resolved source and generation context across the queue in one training record. The record is then converted into the input required by that source, allowing the Generator to continue producing trajectories while the learner consumes prepared inputs. The complete group remains the learning-decision boundary without propagating its completion wait to other rollout generation or model training.

This handoff also preserves the learning roles of the two sources. The decision fixes the sequence, signal, ratio reference, and policy-lag correction represented by the training input. Policy and expert inputs then enter one primary

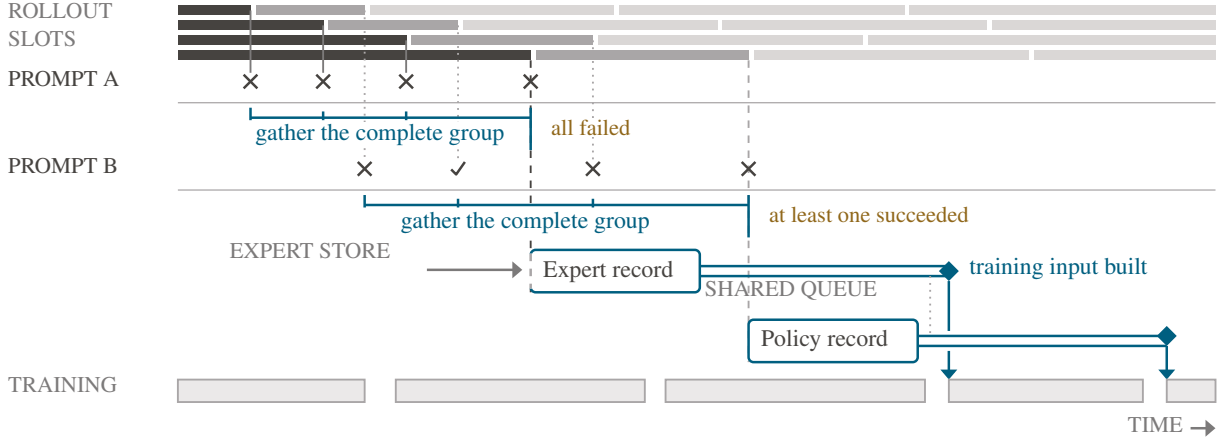


Figure 1: The wait for one complete group is local. While a group is still incomplete, other rollout generation and already-prepared training continue; an all-failure group then selects the expert source and a group containing a success keeps its policy rollouts.

update with one policy objective and the same averaging rule, reducing respectively to group-relative reinforcement and supervised learning. StreamWeave thus confines the required coupling to complete-group decision and training-input construction while leaving the rest of the pipeline open to trajectory-level asynchrony.

Experiments on mathematical reasoning show that StreamWeave reaches an average score of 38.5, competitive among the evaluated RL and expert-trajectory methods and comparable to the 37.7 achieved by a synchronous quality reference with the same group-success selector. The learning dynamics further show that the hard region requiring expert trajectories persists late into training, and that the performance of StreamWeave and an expert-off control separates as training proceeds. StreamWeave sustains this online expert channel even under the longer generation workload observed when expert inputs are used. It reduces the processing time for 128 prompt groups from 46 to 28 seconds and increases prompt-group throughput by $1.64\times$ under the same GPU budget. Together, these results show that group-conditioned learning composition and fully-asynchronous execution can be realized in one training system.

A complete-group decision imposes two requirements whose lifetimes differ. The execution that waits for the group ends once its source is fixed, whereas the information that reproduces the learning role selected by that source must survive until the training input is built. StreamWeave ends each requirement where it is consumed, and our contributions follow that division:

- **Asynchrony-compatible learning composition.** We identify what a resolved source must carry forward for the learner to reproduce the role it selects: the source itself, the reward context of its group, and the generation policy of each rollout. These are expressed as the training input each source requires, and once that input exists no source-specific decision remains in the learner or opti-

mizer path, so both sources share one policy objective and the same averaging rule. A constructive derivation shows that the two inputs reduce to their intended group-relative and supervised contributions.

- **Boundary-localized asynchronous architecture.** We execute trajectory attempts independently, reconstruct the complete group only before its source decision, and preserve the resolved source and generation context until training-input construction. By placing the required processing at the boundaries of the asynchronous queue, StreamWeave retains the basic roles of the Generator and learner and confines complete-group waiting to the source decision.
- **Composition-aware empirical study.** We measure how generation work concentrates where successful policy signal is scarce, how the demand for expert trajectories persists through training, and how expert supervision changes the asynchronous workload. Under these conditions, StreamWeave jointly achieves competitive reasoning performance and $1.64\times$ prompt-group throughput under the same GPU budget.

2 Related Work

StreamWeave connects two lines of work: systems that decouple rollout generation from model training and methods that learn from expert data outside the policy’s own rollouts. The former addresses execution efficiency, while the latter addresses the absence of policy-generated signal. StreamWeave studies their composition when the outcome of a complete group also determines the training source.

Fully-asynchronous policy learning. HybridFlow (Sheng et al. 2025) separates computation and data dependencies in RLHF, providing an execution substrate for flexible algorithms and resource mappings. Asynchronous RLHF (Noukhovitch et al. 2025) directly decouples genera-

tion from learning and studies the trade-off between execution efficiency and policy lag when training on samples from earlier policies. AReaL (Fu et al. 2025) fully disaggregates rollout and training workers and combines workload balancing with staleness-aware optimization in an end-to-end asynchronous LLM RL system. TBA (Bartoldson et al. 2025) trains an off-policy objective on replay experience collected by asynchronous actors. Laminar (Sheng et al. 2026) independently executes trajectories in GRPO to absorb variation in generation latency within a group. These systems decouple the production and consumption of policy-generated rollouts while controlling the resulting policy lag.

Learning from policy rollouts and expert trajectories. A second line of work extends learning beyond experience generated by the current policy through demonstrations, offline data, or expert trajectories. In general reinforcement learning, DQfD (Hester et al. 2018) uses demonstrations in temporal-difference updates and a supervised loss, while RLDP (Ball et al. 2023) continually incorporates offline data into on-line RL. In LLM post-training, InstructGPT (Ouyang et al. 2022) stages demonstration-based supervised learning before RLHF, and SimpleMix (Li and Khashabi 2025) directly mixes on-policy and off-policy data for preference learning. For reasoning post-training, LUFFY (Yan et al. 2025) combines off-policy reasoning traces with policy rollouts through mixed-policy learning. CHORD (Zhang et al. 2026) reformulates SFT as a dynamically weighted auxiliary objective optimized alongside on-policy exploration. ReLIFT (Ma et al. 2026) and SRFT (Fu et al. 2026) also use expert trajectories for reasoning post-training. This line of work has developed methods for selecting external data and assigning a learning role and weight to each source.

Composing asynchronous execution with heterogeneous supervision. Combining these lines requires converting a complete-group decision into a training record that an asynchronous stream can consume. The group outcome chooses between policy rollouts and an expert trajectory, and the chosen source changes both the sequence used for learning and the conditions of its training input. Neither line determines how far the wait for that outcome should reach, nor how long the selected source and its group context must remain available. A system whose groups carry no consequence for the learning signal never faces the first question, and a synchronous barrier that holds every sequence until the update answers the second by default.

3 StreamWeave

StreamWeave separates three units: trajectory attempts for execution, complete prompt groups for learning decisions, and source-appropriate training inputs for optimization. Attempts execute independently; when a group completes, it selects either the policy group or its paired expert trajectory. Cross-attempt waiting ends once the source is fixed, while the source and required group context persist until input construction. Outside this handoff, the Generator continues rollout generation and the learner updates prepared inputs. Section 3.1 asks what a complete-group decision must carry

forward for the learner to reproduce the role it selects. Section 3.2 asks how far the execution that waits for that decision has to reach. Figure 2 shows the three data-state boundaries at the queue handoff, and Algorithm 1 summarizes the concurrent control flow.

3.1 Learning Composition

A complete-group decision fixes more than which sequence to train on: it fixes the conditions under which the learner interprets that sequence. This section identifies what such a decision must carry forward, expresses it as the training input each source requires, and shows that both inputs then enter one policy objective with the same averaging rule. Our experiments instantiate the source rule with Hybrid Post-Training (HPT) (Lv et al. 2025), which selects between a policy group and an expert trajectory from the outcome of a complete rollout group.

Source selection from a completed group. For prompt x , let $G_x = \{\tau_{x,1}, \dots, \tau_{x,n}\}$ denote n policy-generated rollouts. Each rollout has verifier score $R(\tau_{x,i})$, and the success rate of the complete group is

$$P_x = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[R(\tau_{x,i}) > \delta], \quad (1)$$

$$z_x = S_\gamma(G_x) = \begin{cases} \text{expert}, & P_x \leq \gamma, \\ \text{policy}, & P_x > \gamma. \end{cases}$$

Training uses the generated group G_x when $z_x = \text{policy}$ and the expert trajectory τ_x^* paired with the same prompt when $z_x = \text{expert}$. Because the decision is defined on the outcome of G_x rather than on any individual trajectory, the source can be resolved only after all n verifier scores have arrived. Section 4.1 specifies n , γ , and the coverage of expert trajectories.

Constructing source-specific training inputs. The source decision determines not only the sequence used for learning, but also how its learning signal is formed, which reference defines its policy ratio, and whether policy-lag correction applies. For token t in selected sequence r , let $\ell_{\theta,r,t}$ be its log-probability under the current policy, $A_{r,t}$ the learning signal, $\rho_{r,t}$ the policy ratio, and $w_{r,t}$ the importance weight that corrects the difference between the rollout policy and the policy at update entry.

For the policy source, StreamWeave retains all n rollouts and compares their verifier scores to compute the GRPO advantage (Shao et al. 2024) $A_{r,t}^{\text{GRP}}$. A rollout may have been generated by a policy version that predates the update. We therefore use the policy at update entry as the ratio reference and account for the lag from generation through a separate weight:

$$A_{r,t} = A_{r,t}^{\text{GRP}}, \quad w_{r,t} = w_{r,t}^{\text{lag}}, \quad (2)$$

$$\rho_{r,t} = \exp(\ell_{\theta,r,t} - \ell_{r,t}^{\text{entry}}).$$

An expert trajectory is an externally supplied target sequence rather than past experience generated by the policy. Its role is therefore to increase the likelihood of the target

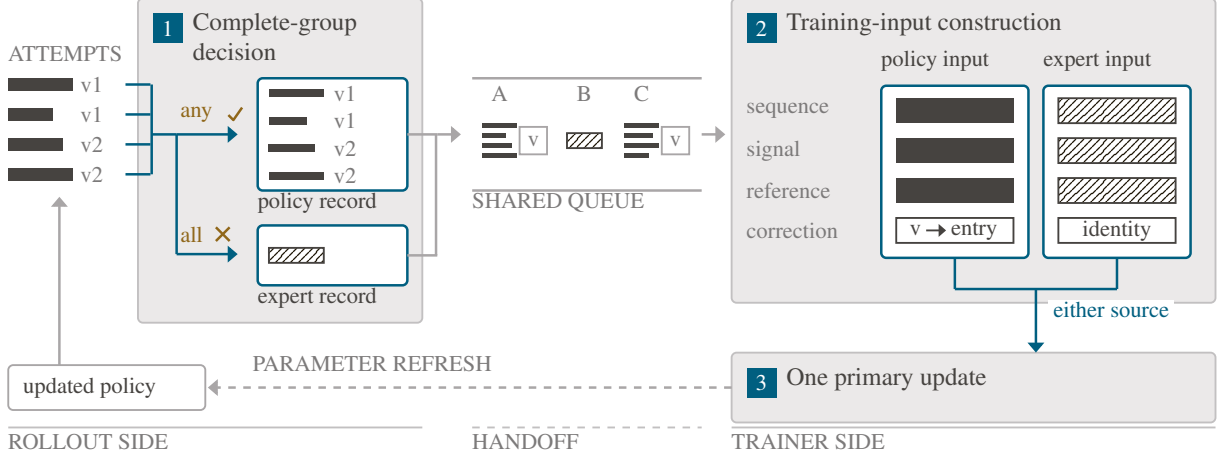


Figure 2: StreamWeave’s three data-state boundaries under the $\gamma = 0$ source rule. A complete group becomes either an n -sequence policy record or a singleton expert record; both share one queue, are converted to source-appropriate inputs, and enter one primary update. Policy provenance enables rollout-to-entry correction, whereas expert input uses identity correction.

directly, without a group-relative signal or rollout-to-entry correction. Using the supervision strength β_r as its signal and a stop-gradient copy of the current-policy log-probability as its reference makes the forward ratio equal to one:

$$\begin{aligned} A_{r,t} &= \beta_r, & w_{r,t} &= 1, \\ \rho_{r,t} &= \exp(\ell_{\theta,r,t} - \text{sg}[\ell_{\theta,r,t}]) = 1. \end{aligned} \quad (3)$$

One policy objective. Inputs from both sources enter the same token-level policy objective Φ . For batch B ,

$$\mathcal{L}(B) = \frac{1}{|B|C} \sum_{r \in B} \sum_t m_{r,t} \Phi(\rho_{r,t}; A_{r,t}, w_{r,t}), \quad (4)$$

where $m_{r,t}$ is the response mask and C is a normalization constant shared by all sequences. The policy input produces a group-relative policy contribution. For the expert input, the forward ratio is one but its numerator retains a gradient, yielding a supervised negative-log-likelihood contribution weighted by β_r . Both inputs therefore share one optimizer path, one policy objective, and the same averaging rule. Our experiments instantiate Φ with vanilla clipped PPO (Schulman et al. 2017). Appendices A.2 and A.3 derive the expert endpoint and the shared averaging rule, establishing that the source semantics are fixed during input construction while the objective and aggregation remain common.

The complete-group decision jointly determines the sequence used for learning and how that sequence contributes to the update. Nothing in this account says how long execution must wait for that decision, which is the question the next section takes up.

3.2 Fully-Asynchronous Execution

Fully-asynchronous execution uses independent attempt scheduling, a bounded queue, backpressure, and parameter refresh to keep rollout generation and model training concurrent. StreamWeave concentrates the composition-specific

Algorithm 1 StreamWeave

Require: Prompt pool $\mathcal{X}$, group size n , verifier R , source rule S_γ , expert store $\mathcal{E}$

- 1: Initialize accumulator $\mathcal{A}$ and queue $\mathcal{Q}$
- 2: **Generator and trainer run concurrently**
- 3: **Generator:** launch an attempt in each idle rollout slot
- 4: On completion of (g_x, i) under π_i^{gen} , store $\langle \tau_i, R(\tau_i), \pi_i^{\text{gen}} \rangle$ in $\mathcal{A}[g_x][i]$
- 5: Immediately launch the next attempt in the released slot
- 6: **if** $|\mathcal{A}[g_x]| = n$ **then**
- 7: $G_x \leftarrow \text{ordered_pop}(\mathcal{A}[g_x]); z_x \leftarrow S_\gamma(G_x)$
- 8: Push $\text{SourceRecord}(x, z_x, G_x, \mathcal{E}[x])$ to $\mathcal{Q}$
- 9: **end if**
- 10: **Trainer:** dequeue source-fixed records and initialize B
- 11: **for each record** q **do**
- 12: Add $\text{PolicyInputs}(q)$ if q is policy, otherwise add $\text{ExpertInput}(q)$
- 13: **end for**
- 14: $\theta \leftarrow \text{LearnerUpdate}(\theta, \mathcal{L}(B));$ publish θ

processing at two boundaries of this handoff: it reconstructs the complete group and resolves the source before the queue, then converts the source-fixed record into the corresponding training input after the queue. Outside these boundaries, the Generator produces trajectory attempts and the learner applies its existing policy update to prepared inputs.

Execute attempts independently and reconstruct each group locally. Trajectory attempts from one prompt group execute independently while retaining their group identity and position. Each completion releases its rollout slot for the next task and enters the accumulator. Once every attempt has arrived, the accumulator restores the complete group in its original order for the source decision. Other groups con-

tinue generating and ready inputs continue training while this group is incomplete, so the complete-group requirement governs decision availability rather than trajectory execution.

Fix the source before the queue. Once the group is complete, StreamWeave applies S_γ from Section 3.1 and resolves whether training will use the policy rollout group or the paired expert trajectory. It sends the resolved source, selected data, and required context through the queue in one prompt-group record: policy records retain complete-group scores and per-rollout generation policies, whereas expert records retain the selected target and its source. Because the source is fixed before transport, queue consumption order can change update timing but cannot revise the learning decision.

Construct training inputs after the queue. The trainer converts each dequeued record into the input defined in Section 3.1. Policy records provide the complete-group scores and provenance needed for the GRPO signal and policy-lag correction; expert records provide the target needed for a β -weighted supervised contribution. The trainer applies the policy objective and averaging rule from Section 3.1 to both inputs.

Keep one asynchronous flow. Source-fixed records remain in the existing queue, backpressure, and parameter-refresh flow, while both inputs share one learner and optimizer path. Waiting for a complete group ends when the accumulator fixes its source; for the policy route, scores and generation context remain in the record until input construction. This handoff preserves complete-group decisions and source-specific learning roles without changing the core interfaces of the surrounding asynchronous flow. Appendix B traces where group context and source identity are consumed and shows how the remaining asynchronous substrate stays on one execution path.

4 Experiments

We evaluate whether StreamWeave preserves the distinct learning roles of policy rollouts and expert trajectories while jointly achieving learning effectiveness and execution efficiency. Our experiments ask three questions. First, does StreamWeave achieve competitive mathematical reasoning performance while retaining cross-domain reasoning? Second, do signal scarcity and generation work concentrate in the all-failure groups that select expert trajectories, and does selective expert supervision contribute to learning beyond those directly routed prompts? Third, can StreamWeave process the generation workload and source-dependent training inputs of persistent expert supervision at their required boundaries, improving prompt-group throughput under the same GPU budget without turning complete-group decisions into a pipeline-wide execution barrier?

4.1 Experimental Setup

Training setting and control. StreamWeave starts from Qwen2.5-Math-1.5B (Yang et al. 2024) and trains on prompts paired with verified expert trajectories from OpenR1-Math (Hugging Face 2025). For each prompt, the policy generates $n = 8$ rollouts. We set the complete-group source

threshold to $\gamma = 0$, so the paired expert trajectory is used only when all eight policy rollouts fail. To isolate the role of the expert channel and the accompanying generation workload, we compare against Async RL (expert-off), which disables the use of expert trajectories while retaining the same fully-asynchronous execution stack. We refer to it as the expert-off control below. Appendices A.2–A.4 and C.1 specify the expert-signal scale, asynchronous correction, loss averaging, and optimization configuration.

Reasoning evaluation. We perform fixed-checkpoint evaluation on six mathematical reasoning benchmarks: AIME24, AIME25, AMC (83 problems), MATH500 (Hendrycks et al. 2021; Lightman et al. 2024), Minerva (Lewkowycz et al. 2022) (272 problems), and Olympiad (He et al. 2024) (674 problems). For AIME24, AIME25, and AMC, we report mean@32, the average pass@1 over 32 stochastic generations. For the remaining benchmarks, we report mean@8. Math Avg. is an equally weighted macro-average of the six unrounded scores. Training reward and our mathematical evaluations use the same Math-Verify grader (Kydliček 2025). We separately evaluate the same checkpoints on ARC-Challenge (Clark et al. 2018), GPQA-Diamond (Rein et al. 2024), and MMLU-Pro (Wang et al. 2024), reporting their macro-average as cross-domain reasoning performance. Baselines include the base and instruct models, Async RL (expert-off), and methods that use RL or expert trajectories (Liu et al. 2025; Yan et al. 2025; Fu et al. 2026; Ma et al. 2026). HPT (sync) is a synchronous quality reference that selects expert trajectories with the same all-failure rule. Results quoted from the original source are marked with † and separated from our evaluations. Appendix C.2 records sampling, rounding, and checkpoint provenance, establishing that Table 1 reports fixed-checkpoint rather than selected-peak evaluation.

Learning-process analysis. We study where the expert channel is needed and how it relates to learning at three levels. First, a generator-side census partitions complete generated groups into all-failure and any-success outcomes and compares their frequency, response-token volume, rollout length, and within-group generation-time spread. Second, we jointly compare routing history, intermediate-checkpoint performance, and the held-out all-failure rate between StreamWeave and the expert-off control. Third, we match the same prompt by the number of successes at its first encounter and compare later encounters, testing whether the observed difference is confined to prompts that directly received an expert input.

Efficiency evaluation. The execution analysis uses two references for different questions. The expert-off comparison measures how using expert trajectories changes generation workload and the incidence of groups containing rollouts from adjacent policy versions within the same fully-asynchronous stack. Synchronous execution under the same GPU budget provides the end-to-end reference. Because a policy-routed group and an expert-routed group produce different numbers of learner samples, we use the unique prompt group before routing as the common unit of work rather

Model	AIME24	AIME25	AMC (83)	MATH500	Minerva	Olympiad	Avg.
Base	6.6	3.5	31.2	43.3	10.9	24.9	20.1
Instruct	10.6	9.4	47.0	75.5	29.5	40.4	35.4
SFT [†]	11.7	13.2	37.8	70.6	26.8	31.3	31.9
RL-only [†]	11.8	7.7	40.2	61.8	26.8	32.0	30.1
Async RL (expert-off)	12.9	7.9	44.9	75.8	28.8	39.5	35.0
HPT (sync)	15.4	12.6	45.8	78.0	31.3	43.2	37.7
SRFT	12.3	10.4	43.0	71.6	26.1	38.4	33.7
ReLIFT	12.6	8.1	40.3	74.6	28.6	39.4	34.0
Oat-Zero	17.2	12.6	49.6	73.7	30.1	38.1	36.9
LUFFY	15.1	14.0	46.0	77.5	30.0	43.5	37.7
StreamWeave	16.1	13.0	47.0	78.5	33.0	43.2	38.5

Table 1: Fixed-checkpoint mathematical reasoning. Scores are rounded to one decimal; Avg. is the unweighted mean of unrounded scores. [†]Source-reported result.

than learner rows or optimizer steps. Throughput is the number of consumed prompt groups divided by training-loop wall-clock, excluding validation and checkpointing. We also measure GPU activity, prompt-group work accumulated over a matched wall-clock interval, and the handling of source-dependent sample counts under a fixed prompt-group input budget. Appendix C.4 defines the hardware allocation, run identifiers, work-weighted throughput estimator, and telemetry normalization, and uses equal-work and sensitivity checks to verify the matched-budget interpretation.

4.2 Learning Effectiveness

Overall mathematical reasoning performance. Table 1 shows that moving the learning composition into fully-asynchronous execution does not forfeit mathematical reasoning performance. StreamWeave reaches 38.5, retaining the 37.7 level of HPT (sync) and remaining competitive among the evaluated RL and expert-trajectory methods.

A costly hard region. An all-failure group, which selects the expert source under our 0/8 rule, provides no reward-derived group-relative signal to the policy branch. We use the generator-side census in Table 2 to test whether this region also imposes greater generation work.

Metric	Any success	All failure
Group frequency	77.6%	22.4%
Generated-token share	73.2%	26.8%
Mean rollout length (tokens)	1,347	1,713
Completion spread (s)	7.7	11.0

Table 2: Generator-side policy rollouts before routing. Completion spread is the within-group difference between the slowest and fastest of eight requests.

All-failure groups account for a larger share of generation than their frequency and exhibit longer, more uneven rollouts. The absence of policy-generated signal therefore coincides with greater generation workload and completion imbalance. Long and uneven generation motivates asynchronous execution, while the 0/8 outcome motivates an expert source.

StreamWeave sustains both requirements in one continuous training system.

Expert routing falls rapidly as the policy improves early in training and then settles into a persistent tail, as shown in Figure 3. The required range of supervision narrows, but a residual region without policy-generated signal remains late in training. This changing use of the expert source is continually incorporated into the same fully-asynchronous training stream as policy rollouts.

StreamWeave and the expert-off control share the execution stack, policy objective, optimizer, prompt population, and expert access; they differ in whether the expert input selected by an all-failure outcome is consumed. Because such a group supplies no reward-derived relative direction, the expert-off run can reach this region only through transfer from other prompts. The runs improve similarly at first and then separate, with the same direction appearing in both average benchmark performance and the held-out all-failure rate.

After matching the same prompt and its initial number of successes, StreamWeave obtains higher subsequent performance in all nine strata $k = 0, \dots, 8$, with the largest gaps in the partial-success region. The pattern is consistent with selective expert contributions supporting both the paired prompt and broader acquisition and retention in the shared policy. Appendix C.3 reports the full stratified results, uncertainty, and later encounters, showing that the pattern extends beyond directly routed prompts while limiting the evidence to a matched longitudinal comparison.

Cross-domain reasoning. To test whether math-focused selective expert supervision narrows broader reasoning, we evaluate the same checkpoints on ARC-Challenge, GPQA-Diamond, and MMLU-Pro.

StreamWeave matches the aggregate cross-domain performance of Async RL (expert-off) and HPT (sync), with individual scores remaining within their range, while achieving stronger mathematical reasoning. The additional learning gain is therefore concentrated in the target mathematical domain without a clear reduction in cross-domain reasoning.

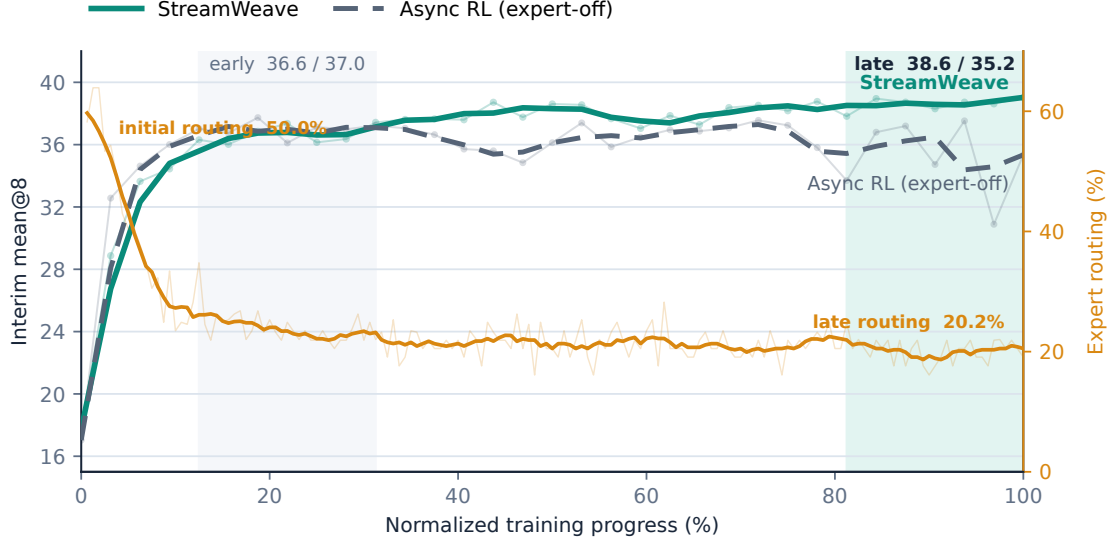


Figure 3: Learning dynamics over normalized training progress. Lines compare interim six-benchmark performance for StreamWeave and the expert-off control; the secondary curve shows expert-routing rate. Thin curves are observations, thick curves are smoothed trends, and shaded regions mark the reported early and late windows.

Model	ARC-C	GPQA-D	MMLU-Pro	Avg.
Base	5.7	3.6	3.3	4.2
Instruct	54.6	27.7	30.3	37.5
Async RL				
(expert-off)	60.8	30.6	31.7	41.1
HPT (sync)	60.5	28.8	33.9	41.1
SRFT	48.0	24.2	28.0	33.4
ReLIFT	52.8	24.5	30.2	35.8
Oat-Zero	41.9	13.5	21.5	25.6
LUFFY	56.5	27.0	34.5	39.3
StreamWeave	60.2	30.3	33.2	41.2

Table 3: Fixed-checkpoint cross-domain reasoning; Avg. is the unweighted mean.

4.3 Execution Efficiency with Expert Supervision

Because the expert source remains part of the online stream throughout training, its accompanying generation workload and group-completion pattern are persistent execution conditions for the fully-asynchronous system. The question for this section is whether those conditions force the complete-group decision to constrain the rest of the pipeline.

Compared with the expert-off control on the same fully-asynchronous stack, the expert-supervised run produces longer responses and group tails, increasing the incidence of groups containing rollouts from adjacent policy versions (Table 4).

This temporal exposure is not uniform across group outcomes. Within StreamWeave, 10.0% of the all-failure groups ultimately assigned to the expert source contain rollouts from adjacent policy versions, the highest rate among the routes.

Metric	Ref.	Ours	Change
<i>Generation workload (expert-off reference)</i>			
Response length (tokens)	896	1,429	1.59×
Longest rollout/group (s)	6.59	12.79	1.94×
Adjacent-version groups	2.03%	7.00%	3.46×
<i>End-to-end execution (synchronous reference)</i>			
128-group time (s) ↓	46.0	28.0	−39%
Throughput (groups/s) ↑	2.78	4.58	1.64×

Table 4: Generation workload and execution efficiency. The synchronous reference uses the same GPU budget; all metrics use pre-routing prompt groups as the work unit.

The region that requires an expert source therefore interacts most strongly with asynchronous execution not only through its missing learning signal, but also through when its complete group becomes available.

When all rollouts in a long and uneven group share one execution phase, the slowest trajectory governs pipeline progress. In the synchronous trace, requests complete in 7.6 seconds on average, but the slowest lasts 23.7 seconds and the generation phase, including post-processing, ends at 25.1 seconds. Earlier completions cannot launch the next attempt before the phase closes, and model training starts only after generation. The completion tail therefore constrains both subsequent rollout work and training.

StreamWeave instead refills each rollout slot as soon as an attempt completes. It preserves group identity and per-rollout generation policy across these independent completions and reconstructs the group only for its source decision. Other groups continue generating and ready data continue training

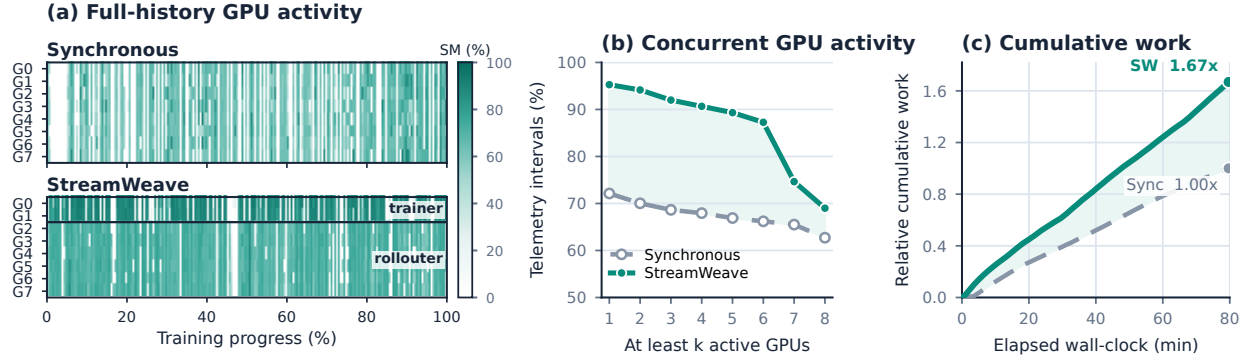


Figure 4: GPU activity and cumulative work under the same 8-GPU budget. Panels (a)–(b) independently normalize each run’s non-validation history; panel (c) compares the complete synchronous run with an equal-length StreamWeave prefix, normalized to the synchronous endpoint.

while a group remains incomplete. The required coordination therefore ends at the decision and input-construction boundaries rather than propagating into pipeline-wide waiting.

Figure 4 shows the hardware-level signature of this design: StreamWeave reduces globally low-activity intervals, sustains broader concurrent GPU activity, and accumulates more prompt-group work over the same wall-clock interval. This connects the bounded scope of complete-group waiting to realized work.

StreamWeave reduces the processing time for 128 prompt groups from 46 to 28 seconds and achieves $1.64\times$ **full-history prompt-group throughput** under the same GPU budget. Two connected execution effects contribute to this end-to-end gain. First, model training that follows generation in synchronous execution proceeds concurrently with rollout generation. Second, immediately refilling a rollout slot after an attempt finishes recovers capacity that would otherwise be lost while early rollouts wait for the complete-group tail. Together, these effects retain the complete-group decision while reducing how far its waiting constrains model training and other rollout slots.

Appendices C.4 and C.5 check this interpretation with resource-normalized accounting, source-specific policy-version composition, and request-level timing, while delimiting it as critical-path accounting rather than an exact additive decomposition.

5 Conclusion

StreamWeave realizes group-conditioned RLVR under fully-asynchronous execution by separating trajectory execution from the complete-group decision that selects the training source and learning role. The wait for that decision ends where the source is fixed, and the information that reproduces the selected role ends where the training input is built. Localizing these dependencies retains the complete group as a learning-decision boundary while separating its completion from a pipeline-wide execution barrier.

Experiments show that all-failure groups carry disproportionately high generation work, demand for the expert

source persists late into training, and selective expert contributions relate to acquisition and retention beyond directly routed prompts. StreamWeave achieves mathematical reasoning performance comparable to the synchronous quality reference and retains competitive cross-domain reasoning. Even under the longer generation workload observed in the expert-supervised run, it improves prompt-group throughput by $1.64\times$ under the same GPU budget. Our direct empirical scope is one backbone, one training seed, an 8-GPU budget, and math-centered group-conditioned two-source RLVR. Within this scope, StreamWeave preserves complete-group learning composition, bounds the execution scope of its dependencies, and combines selective expert supervision with the efficiency of fully-asynchronous execution in one training system.

References

- Ball, P. J.; Smith, L.; Kostrikov, I.; and Levine, S. 2023. Efficient Online Reinforcement Learning with Offline Data. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 1577–1594.
- Bartoldson, B. R.; Venkatraman, S.; Diffenderfer, J.; Jain, M.; Ben-Nun, T.; Lee, S.; Kim, M.; Obando-Ceron, J.; Bengio, Y.; and Kailkhura, B. 2025. Trajectory Balance with Asynchrony: Decoupling Exploration and Learning for Fast, Scalable LLM Post-Training. In *Advances in Neural Information Processing Systems*.
- Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Tafford, O. 2018. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457.
- Fu, W.; Gao, J.; Shen, X.; Zhu, C.; Mei, Z.; He, C.; Xu, S.; Wei, G.; Mei, J.; Wang, J.; Yang, T.; Yuan, B.; and Wu, Y. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. In *Advances in Neural Information Processing Systems*, volume 38, 36256–36282. Curran Associates, Inc.

- Fu, Y.; Chen, T.; Chai, J.; Wang, X.; Tu, S.; Yin, G.; Lin, W.; Zhang, Q.; Zhu, Y.; and Zhao, D. 2026. SRFT: A Single-Stage Method with Supervised and Reinforcement Fine-Tuning for Reasoning. In *International Conference on Learning Representations*.
- He, C.; Luo, R.; Bai, Y.; Hu, S.; Thai, Z. L.; Shen, J.; Hu, J.; Han, X.; Huang, Y.; Zhang, Y.; Liu, J.; Qi, L.; Liu, Z.; and Sun, M. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3828–3850. Association for Computational Linguistics.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. In *Advances in Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Hester, T.; Vecerik, M.; Pietquin, O.; Lanctot, M.; Schaul, T.; Piot, B.; Horgan, D.; Quan, J.; Sendonaris, A.; Osband, I.; Dulac-Arnold, G.; Agapiou, J.; Leibo, J. Z.; and Gruslys, A. 2018. Deep Q-learning From Demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3223–3230.
- Hugging Face. 2025. Open R1: A fully open reproduction of DeepSeek-R1. <https://github.com/huggingface/open-r1>. OpenR1-Math-220k dataset: <https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>.
- Kydlíček, H. 2025. Math-Verify: Math Verification Library. <https://github.com/huggingface/Math-Verify>.
- Lewkowycz, A.; Andreassen, A.; Dohan, D.; Dyer, E.; Michalewski, H.; Ramasesh, V.; Slone, A.; Anil, C.; Schlag, I.; Gutman-Solo, T.; Wu, Y.; Neyshabur, B.; Gur-Ari, G.; and Misra, V. 2022. Solving Quantitative Reasoning Problems with Language Models. In *Advances in Neural Information Processing Systems*, volume 35.
- Li, T.; and Khashabi, D. 2025. SimpleMix: Frustratingly Simple Mixing of Off- and On-policy Data in Language Model Preference Learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 35081–35100.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s Verify Step by Step. In *International Conference on Learning Representations*.
- Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W. S.; and Lin, M. 2025. Understanding R1-Zero-Like Training: A Critical Perspective. In *Second Conference on Language Modeling*.
- Lv, X.; Zuo, Y.; Sun, Y.; Liu, H.; Wei, Y.; Chen, Z.; Zhu, X.; Zhang, K.; Wang, B.; Ding, N.; and Zhou, B. 2025. Towards a Unified View of Large Language Model Post-Training. arXiv:2509.04419.
- Ma, L.; Liang, H.; Qiang, M.; Tang, L.; Ma, X.; Wong, Z. H.; Niu, J.; Shen, C.; He, R.; Li, Y.; Cui, B.; and Zhang, W. 2026. Learning What Reinforcement Learning Can’t: Interleaved Online Fine-Tuning for Hardest Questions. In *International Conference on Learning Representations*.
- Noukhovitch, M.; Huang, S.; Xhonneux, S.; Hosseini, A.; Agarwal, R.; and Courville, A. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. In *International Conference on Learning Representations*.
- Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C. L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; Schulman, J.; Hilton, J.; Kelton, F.; Miller, L.; Simens, M.; Askell, A.; Welinder, P.; Christiano, P. F.; Leike, J.; and Lowe, R. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, volume 35, 27730–27744.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. In *First Conference on Language Modeling*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Sheng, G.; Tong, Y.; Wan, B.; Zhang, W.; Jia, C.; Wu, X.; Wu, Y.; Li, X.; Zhang, C.; Peng, Y.; Lin, H.; Liu, X.; and Wu, C. 2026. Laminar: A Scalable Asynchronous RL Post-Training Framework. In *Proceedings of the 21st European Conference on Computer Systems*, EuroSys ’26, 400–422. New York, NY, USA: Association for Computing Machinery.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, 1279–1297. New York, NY, USA: Association for Computing Machinery.
- Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; Li, T.; Ku, M.; Wang, K.; Zhuang, A.; Fan, R.; Yue, X.; and Chen, W. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Yan, J.; Li, Y.; Hu, Z.; Wang, Z.; Cui, G.; Qu, X.; Cheng, Y.; and Zhang, Y. 2025. Learning to Reason under Off-Policy Guidance. In *Advances in Neural Information Processing Systems*.
- Yang, A.; Zhang, B.; Hui, B.; Gao, B.; Yu, B.; Li, C.; Liu, D.; Tu, J.; Zhou, J.; Lin, J.; Lu, K.; Xue, M.; Lin, R.; Liu, T.; Ren, X.; and Zhang, Z. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv:2409.12122.
- Zhang, W.; Xie, Y.; Sun, Y.; Chen, Y.; Wang, G.; Li, Y.; Ding, B.; and Zhou, J. 2026. On-Policy RL Meets Off-Policy Experts: Harmonizing Supervised Fine-Tuning and Reinforce-

ment Learning via Dynamic Weighting. In *International Conference on Learning Representations*.